

第3章

jQuery设计基础

为了简化JavaScript开发，设计人员编写了一些JavaScript代码库。这些代码库封装了很多预定义的对象和实用函数，能够简化开发人员的工作，提高代码执行效率。可以说，jQuery改变了用户编写JavaScript程序的方式。

3.1 jQuery概述

3.1.1 jQuery简介

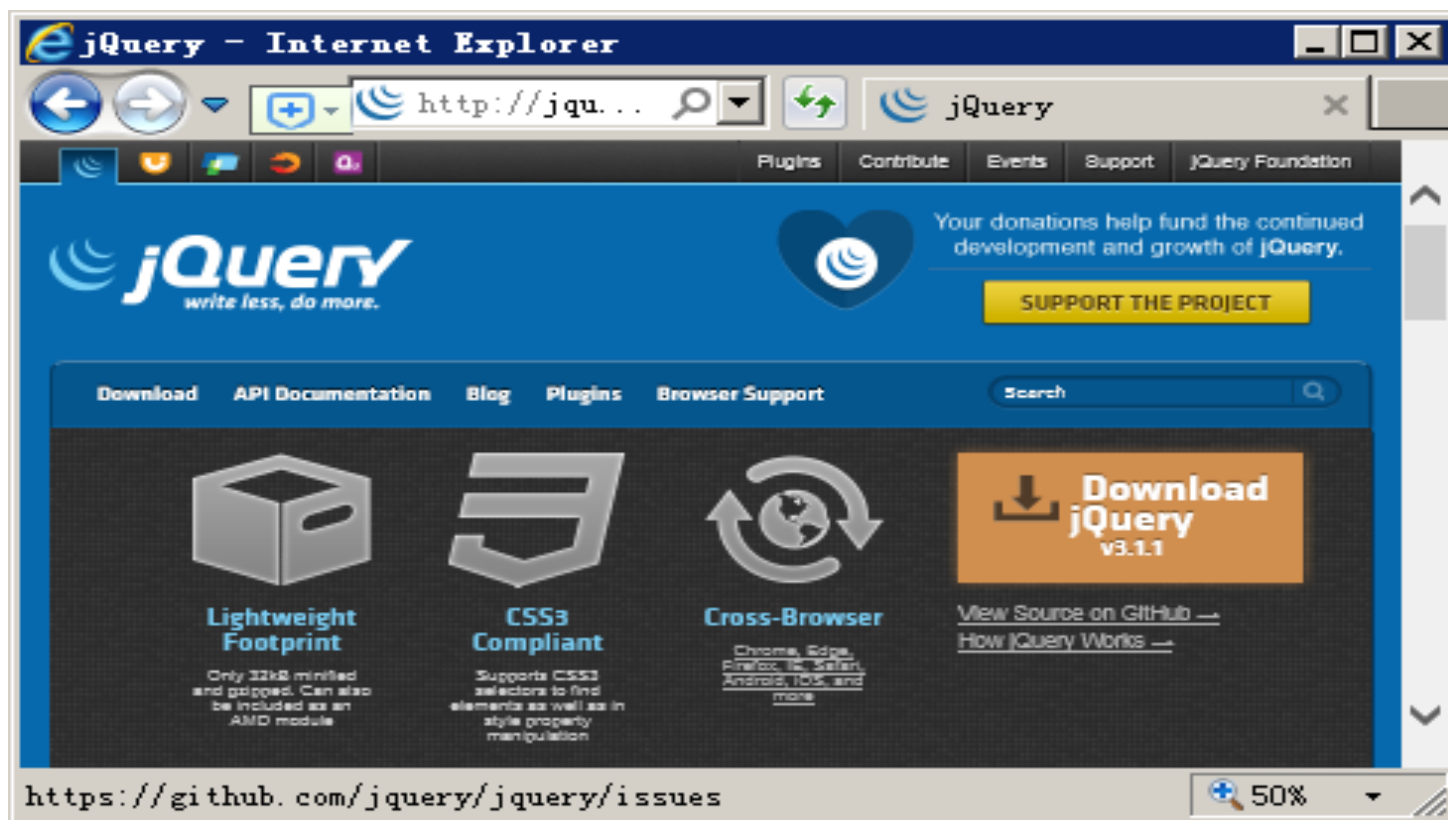
1. 认识jQuery

jQuery是一套JavaScript函数库，使用jQuery可以轻松访问网页元素、修改网页外观与内容、显示动画和响应用户的输入。使用jQuery，可以简化JavaScript编程，通常需要编写几十行甚至更多的JavaScript代码才能实现的功能，使用jQuery，只需要简单的几行甚至一行代码就可以搞定。

2. jQuery的下载地址

jQuery的官方网站（<http://jquery.com>）提供了jQuery最新版本的下载，jQuery的官方网站提供了二个下载的版本：

jquery-3.1.1.js（未压缩版）、jquery-3.1.1.min.js（压缩版），通常情况下，只要使用jquery-3.1.1.min.js（压缩版）就可以了。



3. 如何使用jQuery

在JavaScript程序中使用jQuery有两种方式，一种是下载jQuery库的js文件到本地计算机上，另一种是使用CDN（Content Delivery Network，网络分发）。

（1）使用下载的jQuery的js文件

通常引用语句应该位于<head>部分，代码如下：

```
<head>  
  <script type="text/javascript" src="jquery-3.1.1.min.js"></script>  
</head>
```

（2）使用CDN（网络分发）

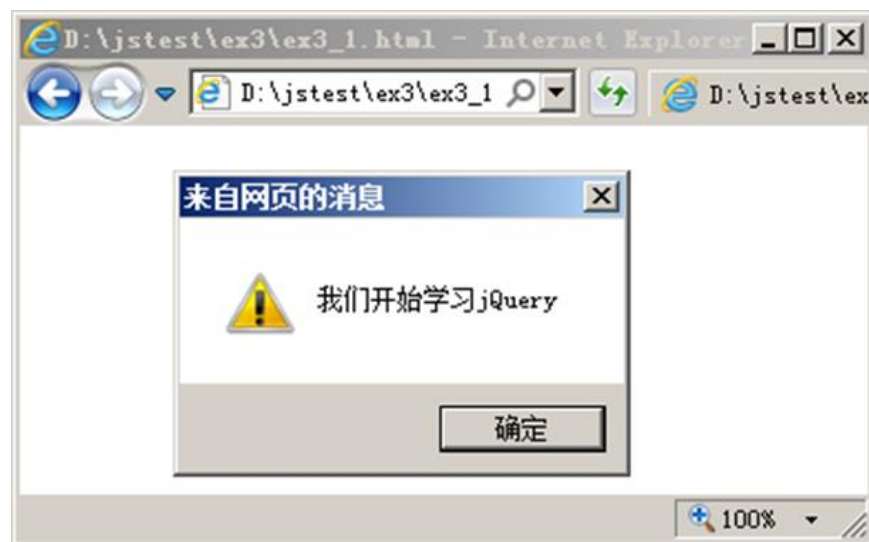
```
<script src="https://code.jquery.com/jquery-3.1.1.min.js"></script>
```

3.1.2 jQuery代码的编写

1. 第一个jQuery程序

[例3—1]一个简单的jQuery程序示例。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
  </head>
  <body>
    <script>
      jQuery(document).ready(function() {
        alert("我们开始学习 jQuery");
      });
    </script>
  </body>
</html>
```



2. jQuery程序的代码功能分析

在例3—1的代码中有一段如下的代码片段：

```
jQuery(document).ready(function(){  
    ..... //程序段  
});
```

这段代码的作用类似于JavaScript中的window.onload方法：

```
window.onload=function(){  
    ..... //程序段  
}
```

虽然上述两段代码在功能上相同，但它们之间又有许多区别：

（1）执行时间不同：

jQuery(document).ready在页面框架下载完毕后就执行；而window.onload必须在页面全部加载完毕后才能执行。很明显前者的执行效率高于后者。

（2）执行数量不同：

jQuery(document).ready可以重复写多个，并且每次执行结果不同；而window.onload尽管可以执行多个，但仅输出最后一个执行结果，无法完成多个结果的输出。

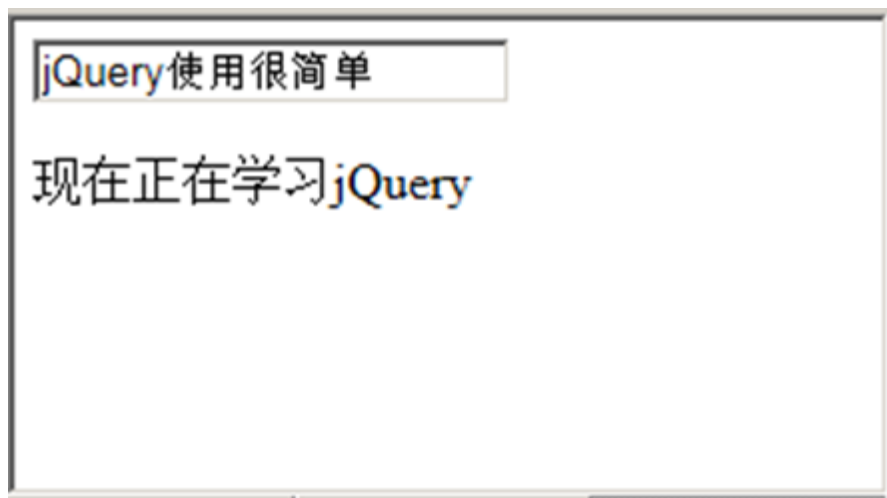
3.2 jQuery方法

1. jQuery的核心方法

- jQuery（）是jQuery体系的核心方法，所有的jQuery操作都在jQuery（）方法中进行。jQuery系统为方法名“jQuery”定义了一个别名“\$”，以避免在程序中出现过多的“jQuery”，便于程序阅读和维护。

- [例3—2] 使用jQuery方法别名的示例，修改页面元素的文字属性。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
    <script>
      $(document).ready(function() {
        $("#aa").val("jQuery 使用很简单");
        $("#bb").text("现在正在学习 jQuery");
      });
    </script>
  </head>
  <body>
    <div>
      <input id="aa" type="text">
      <p id="bb"> jQuery 方法 </p>
    </div>
  </body>
</html>
```



2. ready()方法的简化

在jQuery中，ready()常被简化，ready(function())被简化为一个匿名函数。

代码 `$(document).ready( function(){ .....; } )`

可以简写成: `$( function(){ .....; } )`

这里，`function(){ }`是匿名函数，ready()被简化掉了。

```
jQuery(document).ready(function() {  
    .....    //程序段  
})
```

等价于

```
$(function() {  
    .....    //程序段  
})
```

【例3-3】 用jQuery程序的简化形式重写例3-2的代码。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
    <script>
      $(function() {
        $("#aa").val("jQuery 使用很简单");
        $("#bb").text("现在正在学习 jQuery");
      });
    </script>
  </head>
  <body>
    <div>
      <input id="aa" type="text">
      <p id="bb"> jQuery 方法 </p>
    </div>
  </body>
</html>
```

3.3 jQuery选择器

对页面中的某个元素进行功能操作时，必须先准确地找到该元素。jQuery选择器就是对页面中的元素进行定位的。jQuery选择器分为基本选择器，层次选择器，过滤器选择器和表单选择器。

3.3.1 jQuery的基本选择器

jQuery的基本选择器与CSS选择器语法相同，也是通过元素标签、id、class的方式来指定页面中元素。

表3-1 jQuery的基本选择器

选择器	描述	示例
#id	根据给定的 id 匹配一个元素	<code>\$("#test")</code> 选取 id 为 test 的单个元素
.class	根据给定的类名匹配元素	<code>\$(".test")</code> 选取所有 class 名为 test 的元素
element	根据给定的元素名匹配元素	<code>\$("p")</code> 选取所有的<p>元素
*	匹配所有元素	<code>\$("*")</code> 选取所有的元素
selector1, selector2, …… , selectorN	将每一个选择器匹配到的元素合并后一起返回	<code>\$("div, span, p.myClass")</code> 选取所有 <div>, 和拥有 class 为 myClass 的 <p>标签的一组元素

[例3-4] jQuery基本选择器的#id选择器应用示例。在HTML页面上有一个id属性值为test的文本框和一个按钮，通过单击按钮获取文本输入框中所输入的值。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
    <script>
      jQuery(document).ready(function() {
        $("input[type='button']").click(function() {
          var inputValue = $("#test").val();
          alert(inputValue);
        });
      });
    </script>
  </head>
  <body>
    <input type="text" id="test" value="" />
    <input type="button" value="获取输入值" onclick="" />
  </body>
</html>
```



[例3-5] jQuery基本选择器的.class选择器应用示例。在HTML文档中设置一个空区域块，并由CSS设置该区域的背景颜色，通过jQuery方法改变该区域块的背景颜色。

```
<!doctype html>
<html>
<head>
  <meta charset="UTF-8">
  <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
  <style type="text/css">
    .box{width:300px; height:150px; background-color:#ccc; } ← css 设置区域背景色
  </style>
  <script>
    function fun()
    {
      $(".box").css("background-color", "#33a"); ← jQuery 修改区域背景色
    }
  </script>
</head>
<body>
  <div class="box"></div>
  <input type="button" value="改变颜色" onclick="fun()">
</body>
</html>
```



3.3.2 jQuery的层次选择器

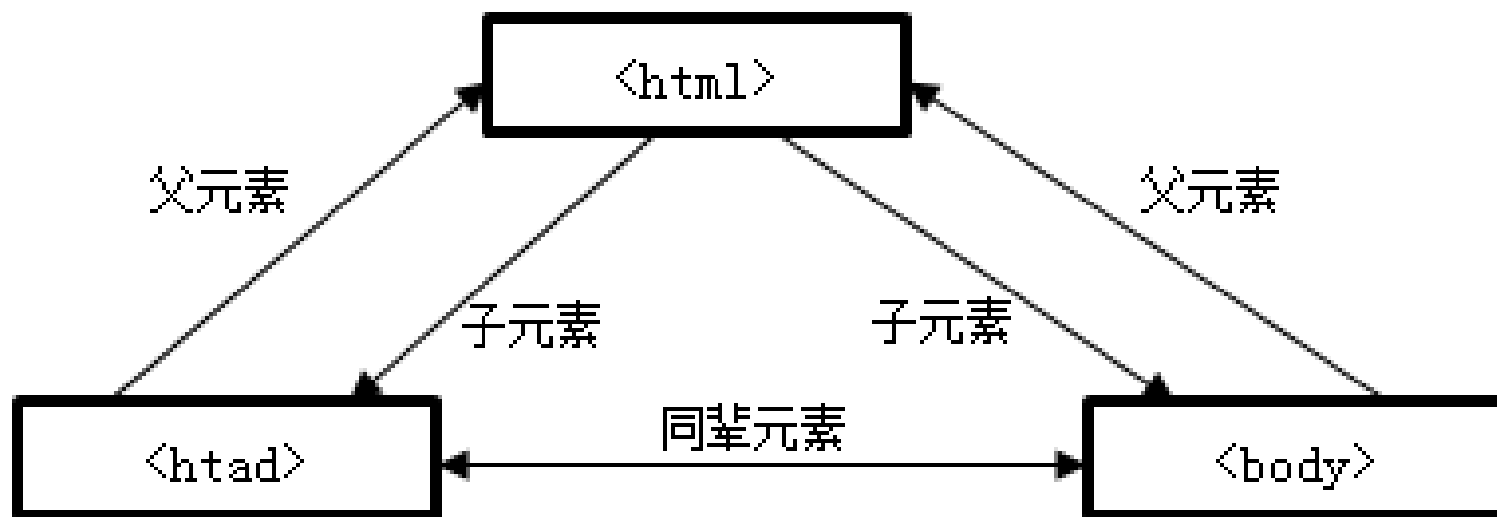


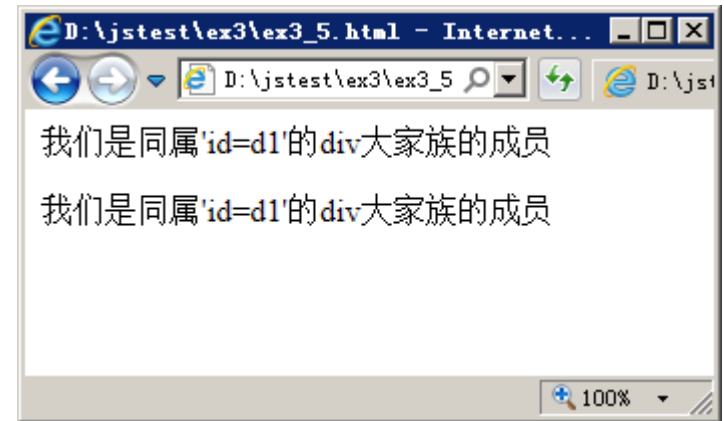
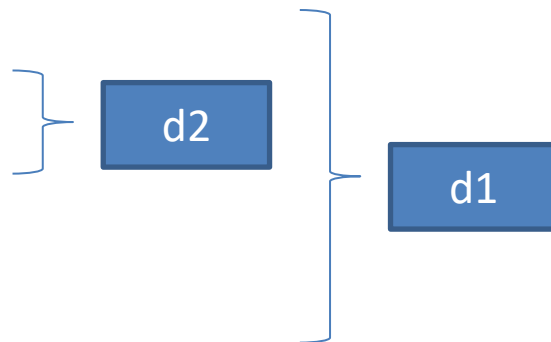
图 3.6 元素的层次关系示意图

表3-2 jQuery的层次选择器

选择器	描述	示例
<code>\$("ancestor descendant")</code>	选取 ancestor 元素里的所有 descendant（后代元素）	<code>\$("div span")</code> 选取<div>里的所有的元素
<code>\$("parent>child")</code>	选取 parent 元素下的 child（子）元素	<code>\$("div > span")</code> 选取<div>元素下元素名是的子元素
<code>\$("prev+next")</code>	选取紧接在 prev 元素后的 next 元素	<code>\$(".one+div")</code> 选取 class 为 one 的下一个<div>元素
<code>\$("prev~siblings")</code>	选取 prev 元素之后的所有 siblings 元素	<code>\$("#two~div")</code> 选择 id 为 two 的元素后面的所有<div>兄弟元素

【例3-6】jQuery的层次选择器应用示例。使用层次选择器，获取<div>元素中的全部元素，并设置它们的内容。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
  </head>
  <body>
    <div id="d1">
      <div id="d2">
        <span></span>
      </div>
      <p>
        <span></span>
      </p>
    </div>
    <script>
      $("div span").html("我们是同属 'id=d1' 的 div 家族成员");
    </script>
  </body>
</html>
```



【例3-7】jQuery的层次选择器应用示例。控制HTML文档各级元素的样式。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
  <script>
    jQuery(document).ready(function() {
      $("div").css("border", "solid 1ps red"); //控制文档中所有 div 元素
      $("div>div").css("margin", "2em");      //控制 div 元素中包含的 div 子元素
      $("div div").css("background", "#ff0"); //控制最外层 div 元素包含的所有 div 元素
      $("div div div").css("background", "#f0f"); //控制第 3 层及其以内的 div 元素
      $("div + p").css("margin", "2em");      //控制 div 相邻的 p 元素
      $("div:eq(1) ~ p").css("background", "cyan"); //控制 div 后面并列的所有 p 元素
    });
  </script>
</head>
```

应为px

```
<body>
  <div>一级 div 元素
    <div> 二级 div 元素
      <div>
        三级 div 元素
      </div>
      <p>段落文本 11</p>
      <p>段落文本 12</p>
    </div>
    <p>段落文本 21</p>
    <p>段落文本 22</p>
  </div>
  <p>段落文本 31</p>
  <p>段落文本 32</p>
</body>
</html>
```

一级div元素

二级div元素

三级div元素

段落文本11

段落文本12

段落文本21

段落文本22

段落文本31

段落文本32

3.3.3 jQuery的过滤选择器

1. 基本过滤选择器

选择器	描 述	返 回	示 例
:first	匹配第一个元素	单个元素	<code>\$("div:first")</code>
:last	匹配最后一个元素	单个元素	<code>\$("span:last")</code>
:even	匹配索引是偶数的元素 索引从 0 开始	集合元素	<code>\$("li:even")</code>
:odd	匹配索引是奇数的元素 索引从 0 开始	集合元素	<code>\$("li:odd")</code>
:eq(index)	匹配索引等于 index 的元素 (索引从 0 开始)	单个元素	<code>\$("input:eq(2)")</code>
:gt(index)	匹配索引大于 index 的元素 (索引从 0 开始)	集合元素	<code>\$("input:gt(1)")</code>
:lt(index)	匹配索引小于 index 的元素 (索引从 0 开始)	集合元素	<code>\$("input:lt(5)")</code>
:header	匹配所有 h1, h2...等 标题元素	集合元素	<code>\$(":header")</code>
:animated	匹配所有正在执行 动画的元素	集合元素	<code>\$("div:animated")</code>

【例3-8】基本过滤选择器应用示例。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
    <script>
      jQuery(document).ready(function() {
        $("li:eq(2)").css("background", "cyan"); //选择第 3 个列表项（从 0 算起）
      });
    </script>
  </head>
  <body>
    <ul>
      <li>C++</li>
      <li>Java</li>
      <li>JavaScript</li>
      <li>Python</li>
    </ul>
  </body>
</html>
```

- C++
- Java
- JavaScript
- Python

2. 内容过滤选择器

选择器	描 述	返 回	示 例
:contains(text)	匹配含有文本内容 text 的元素	集合元素	\$(“p:contains(今天)”)
:empty	匹配不含子元素或文本元素的空元素	集合元素	\$(“p:empty”)
:has(selector)	匹配包含 selector 元素的元素	集合元素	\$(“div:has(span)”)
:parent	匹配含有子元素或文本的元素	集合元素	\$(“div:parent”)

【例3-9】jQuery的内容过滤选择器应用示例。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
    <script>
      //jQuery(document).ready(function() {
        $(function() {
          $("li:contains(' JavaScript')").css("color", "cyan");
        });
      </script>
    </head>
    <body>
      <ul>
        <li>C++</li>
        <li>Java</li>
        <li>JavaScript</li>
        <li>Python</li>
      </ul>
    </body>
  </html>
```

- 
- C++
 - Java
 - JavaScript
 - Python

3.3.4 jQuery的表单选择器

选择器	描 述	返 回	示 例
:input	匹配所有 input, textarea, select, button 元素	集合元素	<code>\$("input")</code>
:text	匹配所有文本框	集合元素	<code>\$(":text")</code>
:password	匹配所有密码框	集合元素	<code>\$(":password")</code>
:radio	匹配所有单选框	集合元素	<code>\$(":radio")</code>
:checkbox	匹配所有所有多选框	集合元素	<code>\$(":checkbox")</code>
:submit	匹配所有提交按钮	集合元素	<code>\$(":submit")</code>
:image	匹配所有图像按钮	集合元素	<code>\$(":image")</code>
:reset	匹配所有重置按钮	集合元素	<code>\$(":reset")</code>
:button	匹配所有按钮	集合元素	<code>\$(":button")</code>
:file	匹配所有上传域	集合元素	<code>\$(":file")</code>

【例3-10】jQuery的表单选择器应用示例。

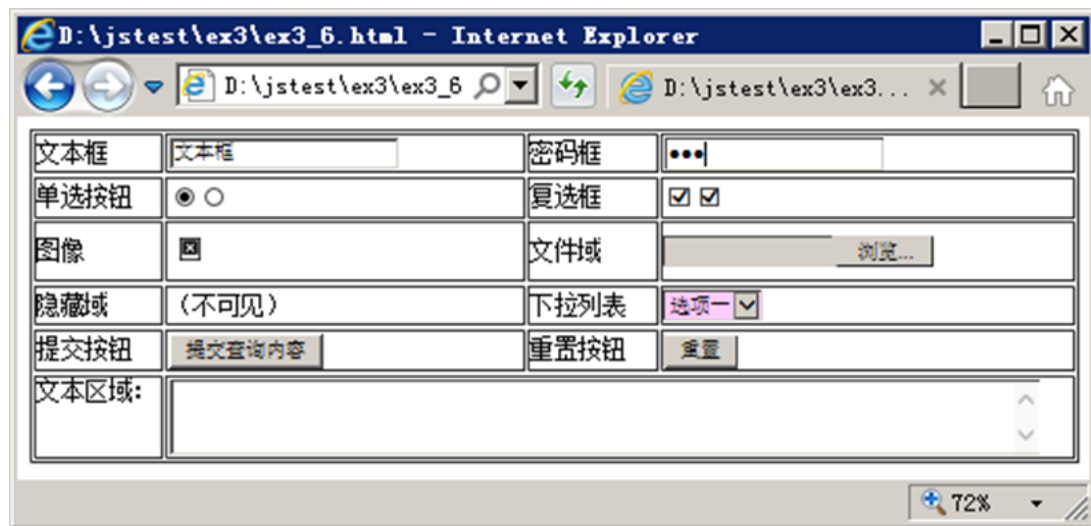
```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <!-- 引入 jQuery -->
  <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
  <script type="text/javascript">
    $(document).ready(function() {
      $(".:text").attr("value", "文本框");
      $(".:password").attr("value", "密码框");
      $(".:radio:eq(0)").attr("checked", "true");
      $(".:checkbox").attr("checked", "true");
      $(".:image").attr("src", "ICO.ICO");
      $(".:file").css("width", "180px");
      $(".:hidden").attr("value", "已保存的值");
      $(".select").css("background", "#FCF"); //注意"select"没有冒号
      $(".:submit").attr("id", "btn1");
      $(".:reset").attr("name", "btn");
      $(".textarea").attr("value", "文本域"); //注意"textarea"没有冒号
    });
  </script>
</head>
```

```
<body>
  <table width="700" height="145" border="1">
    <tr>
      <td width="100" height="23">文本框</td>
      <td width="160"><input type="text"/></td>
      <td width="90">密码框</td>
      <td width="50"><input type="password" /></td>
    </tr>
    <tr>
      <td height="24">单选按钮</td>
      <td><input type="radio" /><input type="radio" /></td>
      <td>复选框</td>
      <td><input type="checkbox" /><input type="checkbox" /></td>
    </tr>
    <tr>
      <td height="36">图像</td>
      <td><input type="image" /></td>
      <td>文件域</td>
      <td><input type="file" /></td>
    </tr>
```

```

<tr>
  <td height="23">隐藏域</td>
  <td><input type="hidden" />（不可见）</td>
  <td>下拉列表</td>
  <td><select><option>选项一</option>
    <option>选项二</option>
    <option>选项三</option>
  </select>
</td>
</tr>
<tr>
  <td height="25">提交按钮</td>
  <td><input type="submit" /></td>
  <td>重置按钮</td>
  <td><input type="reset" /></td>
</tr>
<tr>
  <td valign="top">文本区域: </td>
  <td colspan="3"><textarea cols="70" rows="3"></textarea></td>
</tr>
</body>
</html>

```



3.4 JQuery事件处理

事件处理（Event Handlers）是Jquery程序中一个非常重要的功能，通常用于响应用户的互动或创建特效及动画效果。

3.4.1 事件与事件处理

1. 事件

表 3-6 常见的 DOM 事件

鼠标事件	键盘事件	表单事件	文档/窗口事件
click	keypress	submit	load
dblclick	keydown	change	resize
mouseenter	keyup	focus	scroll
mouseleave		blur	unload

2. jQuery事件处理方法的语法格式

下面以处理点击事件 `click()` 方法来说明 jQuery 事件处理方法的语法格式。

例如，页面中指定一个点击事件：

```
$("p").click();
```

经常在 `$(document).ready()` 中使用一个事件函数来处理事件。`$(document).ready()` 方法是在文档完全加载完后执行的函数。

```
$(document).ready(function() {  
    $("p").click(function() {  
        // 动作触发后执行的代码!!  
    });  
});
```

【例3-11】 处理点击事件click（）方法的示例。

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
    <script>
      $(document).ready(function() {
        $("p").click(function() {
          $(this).hide();
        });
      });
    </script>
  </head>
  <body>
    <p>如果你点我，我就会消失。</p>
    <p>点我消失!</p>
    <p>点我也消失!</p>
  </body>
</html>
```

3.4.2 jQuery的鼠标事件

1. jQuery中常见的鼠标事件

表 3-7 jQuery 中常见的鼠标事件

事件名称	说明
click 事件	单击鼠标左键时触发
dblclick 事件	双击鼠标左键时触发
mousedown 事件	按下鼠标按键时触发（无论左、右键）
mouseup 事件	放开鼠标按键时触发
mouseover 事件	鼠标进入指定元素时触发
mouseout 事件	鼠标移出指定元素时触发
hover 事件	把两个事件连起来使用

2. 鼠标事件示例

(1) click 事件

```
$('#p').click(function() {  
    alert(“运行单击鼠标 click 事件!”);  
});
```

(2) dbclick 事件

```
$('#p').dbclick(function() {  
    alert(“运行双击鼠标 dbclick 事件!”);  
});
```

(3) mousedown 事件

```
$('#p').mousedown(function() {  
    alert('mousedown function is running !');  
});
```

(4) mouseup 事件

```
$('#p').mouseup(function() {  
    alert('mouseup function is running !');  
});
```

(5) mouseover 事件和 mouseout 事件

```
$("#p").mouseover(function() {  
    $("#p").css("background-color", "yellow");  
});  
$("#p").mouseout(function() {  
    $("#p").css("background-color", "#E9E9E4");  
});
```

3. hover()方法

hover()是一个处理多个不同事件的方法，使用该方法时需要2个处理函数作参数。当鼠标进入所选元素时，就执行第1个参数的函数；当鼠标离开所选元素进，则执行第2个参数的函数。

【例3-12】 实现表格的隔行变色，当鼠标悬停时突出显示某行。

姓名	性别	暂住地
张大山	男	福建厦门
李晓丽	女	浙江杭州
王老五	男	湖南长沙
赵顺六	男	广东顺德
吴小芳	男	浙江金华
钱宗保	女	江西九江

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <style type="text/css">
      table{ border:0;border-collapse:collapse;}
      td { font:normal 12px/17px Arial;padding:2px;width:100px;}
      th{ font:bold 12px/17px Arial;text-align:left;
        padding:4px;border-bottom:1px solid #333;}
      .even{ background:#FFF38F;}
      .odd{ background:#FFFFEE;}
      .selected{ background:#FF6500;color:#fff;}
    </style>
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
    <script>
      $(document).ready(function() {
        $("tbody tr:even").addClass("even"); //选择所有相隔的偶数元素(从 0 开始)
        $("tbody tr:odd").addClass("odd");  //选择所有相隔的奇数元素
        $("tbody tr").hover(
          function() { $(this).addClass("selected"); }, //增加颜色加深的样式类
          function() { $(this).removeClass("selected");} //恢复原颜色
        );
      });
    </script>
  </head>
```

```
<body>
  <table>
    <thead>
      <tr><th>姓名</th><th>性别</th><th>籍贯</th></tr>
    </thead>
    <tbody>
      <tr><td>张大山</td><td>男</td><td>福建厦门</td></tr>
      <tr><td>李晓丽</td><td>女</td><td>浙江杭州</td></tr>
      <tr><td>王老五</td><td>男</td><td>湖南长沙</td></tr>
      <tr><td>赵顺六</td><td>男</td><td>广东顺德</td></tr>
      <tr><td>吴小芳</td><td>女</td><td>浙江金华</td></tr>
      <tr><td>钱宗保</td><td>女</td><td>江西九江</td></tr>
    </tbody>
  </table>
</body>
</html>
```

3.4.3 \$.each()方法的循环遍历算法

使用jQuery的\$.each()方法可以遍历任何集合，比如对象或数组。\$.each()方法的第一个参数为要遍历的集合，第二个参数为回调函数，该回调函数每次传递一个数组的下标和这个下标所对应的数组的值。

\$.each()方法的一般格式表示如下：

(1) 遍历数组的一般格式

```
$.each(Array, function(p1, p2) {  
    this;          //这里的 this 指向每次遍历中 Array 的当前元素  
    p1; p2;        //访问附加参数  
}, ['参数 1', '参数 2']);
```

(2) 遍历对象的一般格式

```
$.each(Object, function(name, value) {  
    this;          //this 指向当前属性的值  
    name;          //name 表示 Object 当前属性的名称  
    value;         //value 表示 Object 当前属性的值  
});
```

【例3-13】应用\$.each()方法的循环遍历算法示例。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
  </head>
  <body>
    <div id="id1"></div><br/>
    <div id="id2"></div><br/>
    <div id="id3"></div><br/>
    <script>
      var data1=[1,2,3];
      var data2={name:' 张大山',age:20};
      var data3=[{name:' 张大山',age:20},{name:' 李晓丽',age:21}];
      $.each(data1,function(i,data){
        $("#id1").append(data + " ");
      });
      $.each(data2,function(i,data){
        $("#id2").append(data + "<br/>");

      });
      $.each(data3,
        function(i,data){
          $("#id3").append(data['name'] + " " + data['age'] + "<br/>");
        }
      );
    </script>
  </body>
</html>
```

程序运行结果为:

1 2 3

张大山

20

张大山 20

李晓丽 21

3.5 JQuery自定义插件

3.5.1 JQuery自定义插件规范

JQuery插件实际上就是以Jquery核心代码为基础编写的一段应用程序，开发人员可以使用它创建Web应用程序，提高移动网站的开发效率。

1. JQuery自定义插件的命名规则

如果希望用户在查看文件时立即知道这是什么插件，统一文件名称是必须的。

JQuery自定义插件的命名规则：

jquery.插件名.js

自定义插件名时要注意防止与JavaScript库的插件混淆。

2. 方法返回值

所有jQuery插件的方法都应该返回一个值，除特定需求之外，所有方法都必须返回jQuery对象。

如果需要方法返回计算值或者某个特定对象，一般都应该返回当前上下文环境中的jQuery对象，用this关键字引用。通过这种方式，可以保持jQuery框架方法顺序链的连续。

3. jQuery与\$有区别

在插件代码中总是使用jQuery，而不是使用别名“\$”。

jQuery插件分为jQuery对象级的插件和jQuery类级别的插件，它们在编写形式上有一些差别。

3.5.2 封装jQuery对象级的插件

1. 封装插件

把实现某功能的代码定义为函数，设有函数如下：

```
jQuery.fn.函数名 = function(参数){  
    .....; //实现功能的代码  
};
```

则将该函数封装成插件的语法格式如下：

```
(function($){  
    $.fn. 函数名 = function(参数){  
        .....; //实现功能的代码  
    };  
})(jQuery); //封装插件
```

2. 调用插件

调用插件的语句格式：

`$("#id").函数名（参数）；`

【例3-14】 创建一个能改变对象文字内容功能的插件。

(1) 新建一个名为“jquery.ex3_14.js”的jQuery插件文件，其代码如下：

```
(function($) {  
    $.fn.sayhello = function(name) {    //定义函数名为 sayhello  
        $(this).css({"font-size": "150%"});  
        $(this).text(name + "插件，我来了");  
    };  
})(jQuery);
```

(2) 建立ex3_14.html,调用插件的函数。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <script src="jq/jquery-3.1.1.min.js"></script>
    <script src="jquery.ex3_14.js"></script>    <!-- 引用自定义的插件 -->

    <script>
      $(document).ready(function() {
        $("#sid").click(function() {
          $(this).sayhello("jQuery"); //调用方法 $("id").函数名(参数);
        });
      });
    </script>

  </head>
  <body>
    <h1><p id = "sid">单击我，我会改变 </p></h1>
  </body>
</html>
```

单击我，我会改变

jQuery插件，我来了

3.5.3 定义类级别插件

1. 封闭类级别插件

定义类级别插件的语法格式如下：

```
jQuery.extend({  
    函数名1: function(参数){  
        .....; //实现功能的代码  
    },  
    函数名2: function(参数){  
        .....; //实现功能的代码  
    }  
});
```

2. 调用插件

调用插件的语句格式：

```
$.函数名（参数）；
```

【例3-15】 创建一个类级别的插件。

(1) 新建一个插件文件：jquery.ex3_15.js

```
jQuery.extend({  
    sayhello:function(name) {           //定义函数名为 sayhello  
        return alert(name + "插件，我来了");  
    }  
});
```

(2) 建立 ex3_15.html, 调用插件中的函数。

```
<!doctype html>  
<html>  
  <head>  
    <meta charset="UTF-8">  
    <script src="jq/jquery-3.1.1.min.js"></script>  
    <script src="jquery.ex3_15.js"></script>  <!-- 引用自定义的插件 -->
```

```
<script type="text/javascript">
    $(document).ready(function() {
        $("#sid").click(function() {
            $.sayhello("jQuery");    //调用方法：$.函数名（参数）；
        });
    });
</script>
</head>
<body>
    <h1><p id = "sid">单击我，我会改变 </p></h1>
</body>
</html>
```

【例3-16】编写一个插件程序，点击图片时，显示另一张图片。

(1) 定义插件文件 jquery.ex3_16.js，代码如下：

```
(function($) {  
    $.fn.changeimg=function(img2) {  
        $(this).hide();           //当前图片隐藏  
        $(img2).show();           //另一张图片显示  
    };  
})(jQuery);
```

(2) 建立 ex3_16.html 文件，调用插件函数，代码如下：

```
<!DOCTYPE html>  
<html>  
<head>  
<meta charset="utf-8">  
<script src="jq/jquery-3.1.1.min.js"></script>  
<script src="jquery.ex3_16.js"></script> <!-- 引用自定义的插件 -->
```

```

<script>
$(document).ready(function() {
    $("#imgid1").click(function() {
        $(this).changeimg("#imgid2");
    });
});
function imghide() {
    $("#imgid2").hide();
};
</script>
</head>
<body onload="imghide()">
    <center><h1>
        <p>熊孩子</p>
        
        
    </h1></center>
</body>
</html>

```

熊孩子



点击图片后，
图片改变

熊孩子



【例3-17】实现购物车添加或减少商品自动结算功能的程序设计。

(1) 定义插件文件 jquery.ex3_17.js, 代码如下:

```
jQuery.extend({  
    spadd:function(option) {  
        var t=$(option).parent().find('input[class*=text_box]');  
        t.val(parseInt(t.val())+1);  
    },  
    spmin:function(option) {  
        var t=$(option).parent().find('input[class*=text_box]');  
        t.val(parseInt(t.val())-1)  
        if(parseInt(t.val())<0) {  
            t.val(0);  
        }  
    }  
});
```

(2) 建立 ex3_17.html 文件，调用插件函数，代码如下：

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<script type="text/javascript" src="jquery-3.1.1.min.js"></script>
<script src="jquery.ex3_16.js"></script> <!-- 引用自定义的插件 -->
<script type="text/javascript">
$(function() {
    $(".add").click(function() {
        $.spadd(this); //调用插件的 spadd() 方法
        setTotal();
    })
    $(".min").click(function() {
        $.spmin(this); //调用插件的 spmin() 方法
        setTotal();
    })
})

//计算总价
function setTotal() {
    var s=0;
    $("#tab td").each(function() {
        s += parseInt($(this).find('input[class*=text_box]').val())
            * parseFloat($(this).find('span[class*=price]').text());
    }); // 价格 = 数量 * 单价
    $("#total").html(s.toFixed(2));
}
</script>
```

应为17

```

</head>
<body>
<table id="tab">
  <tr>
    <td>
      <span>单价:</span><span class="price">1.50</span>
      <input class="min" name="" type="button" value="-" />
      <input class="text_box" name="" type="text" value="1" />
      <input class="add" name="" type="button" value="+" /></td>
    </tr>
    <tr>
      <td>
        <span>单价:</span><span class="price">3.95</span>
        <input class="min" name="" type="button" value="-" />
        <input class="text_box" name="" type="text" value="1" />
        <input class="add" name="" type="button" value="+" /></td>
      </tr>
    </table>
    <p>总价:<label id="total"></label></p>
  </body>
</html>

```

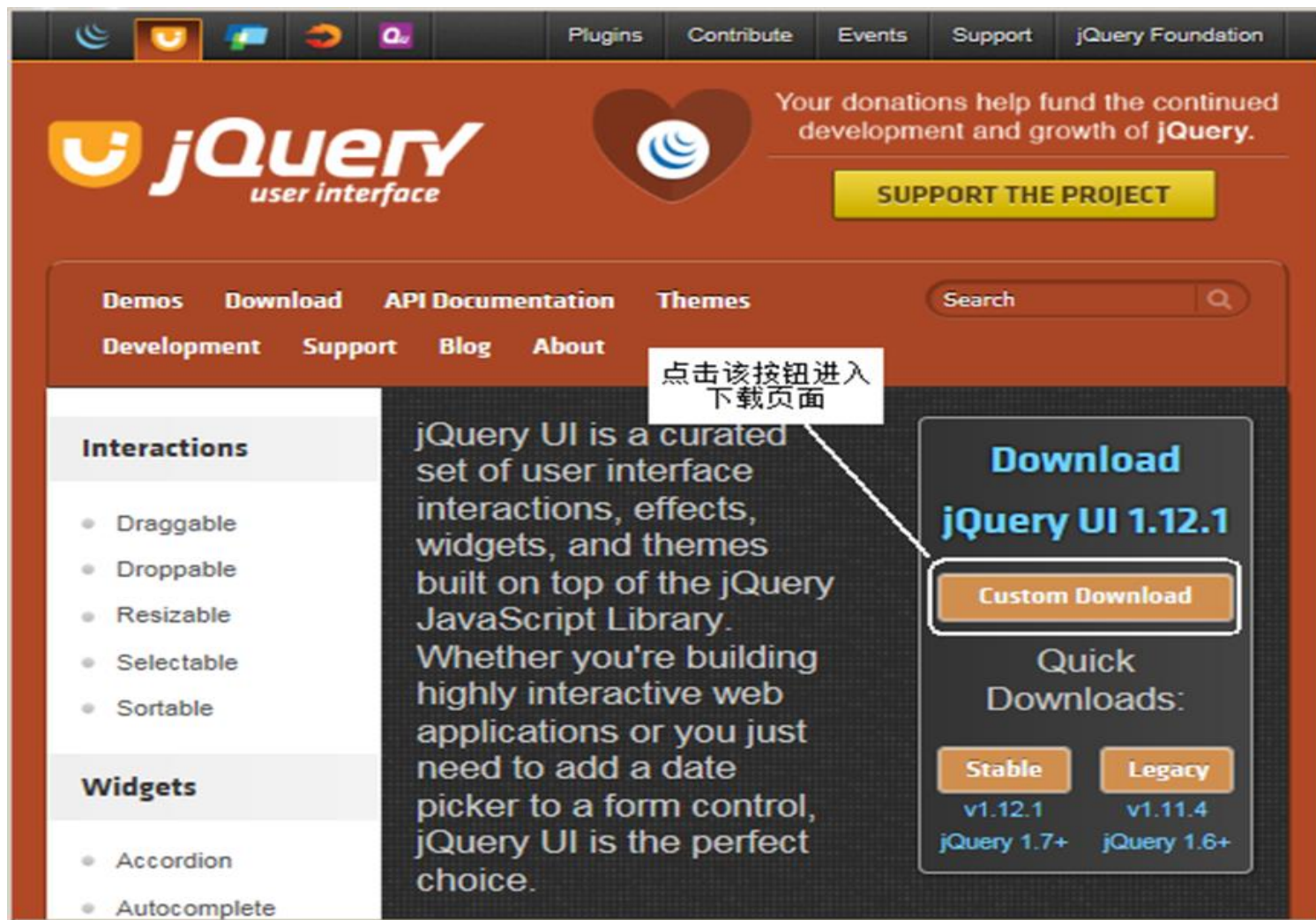
单价:1.50	-	3	+
单价:3.95	-	5	+
总价:24.25			

3.5.4 使用jQuery UI插件

- UI是User Interface的简称，即用户界面。
jQuery UI是以jQuery为基础的开源用户界面代码库，它包含底层用户交互、动画、特效和可更换主题的可视组件。jQuery UI是未来jQuery技术框架发展的趋势，也是未来互联网客户端程序发展的方向。

1. jQuery UI的下载

- 官方网址: “<http://jqueryui.com/>”



2. jQuery UI插件的使用

jQuery UI下载后，得到一个包含所选组件的zip压缩文件jquery-ui-1.12.1.zip，解压该文件到需要使用jQuery UI的应用程序目录下即可。

下面以日期选择器（Datepicker）插件为例说明jQuery UI插件的使用。

日期选择器（Datepicker）插件主要用来从弹出的在线日历中选择一个日期。

(1) 引用下载的 jQuery UI 插件(增加了支持中文的 datepicker_cn.js 引用):

```
<script src="js/jquery-ui.min.js"></script>
<script src="js/datepicker.js"></script>
<script src="js/datepicker-zh-CN.js"></script>
```

(2) 引入默认样式表文件

```
<link rel="stylesheet" href="jquery-ui.min.css" />
<link rel="stylesheet" href="js/jquery.mobile.datepicker.css" />
```

(3) 在 HTML 页面中添加输入框组件, 设置 id 属性值

```
<input type="text" id="selectDate"/>
```

(4) jQuery UI 插件的日期选择器调用

```
$(document).ready(function() {
    $('#selectDate').datepicker();
});
```

【例3-18】 使用日期选择器（Datepicker）插件选择日期示例。

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
<script type="text/javascript" src="jquery.ex3_16.js"></script>
    <!-- 添加 datepicker 支持 -->
    <script src="jquery-ui.min.js"></script>
    <script src="js/datepicker.js"></script>
    <script src="js/datepicker-zh-CN.js"></script>
    <!--引入样式 css-->
    <link rel="stylesheet" href="jquery-ui.min.css" />
    <link rel="stylesheet" href="js/jquery.mobile.datepicker.css" />
    <script type="text/javascript">
      $(document).ready(function() {
        $('#selectDate').datepicker();
      });
    </script>
  </head>
```

```
<body>
  <div id="">
    <center><h3>日期选择器使用示例</h3></center><br />
    选择日期: <input type="text" id="selectDate"/>
  </div>
</body>
</html>
```

日期选择器使用示例

选择日期:

⏪

2017年 七月

⏩

一	二	三	四	五	六	日
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31						

3.6 jQuery动画与特效

3.6.1 jQuery的特效方法

jQuery提供了一些常用的特效：显示、隐藏、滑动和淡入淡出，可以通过所选择元素执行特效方法来创建动画与特效。

1. jQuery特效的基本语法

\$(选择器).特效方法（持续时间， 回调函数）；

表 3-8 jQuery 常用的特效方法

特效名称	特效方法	方法说明
显示与隐藏	show()	显示元素
	hide()	隐藏元素
	toggle()	单击切换显示或隐藏元素
滑动	slideDown()	元素向下滑动
	slideUp()	元素向上滑动
	slideToggle()	切换元素向上或向下滑动
淡入淡出	fadeIn()	淡入元素，即慢慢变成不透明
	fadeOut()	淡出元素，即慢慢变成透明
	fadeToggle()	切换交叉进出元素
	fadeTo()	元素慢慢变成指定的透明度

2. 显示和隐藏元素show()和hide()

对于动画来说，显示和隐藏是最基本的效果之一，jQuery使用show()和hide()方法显示和隐藏HTML元素，创建基本的特效。

show()和hide()方法的语法如下：

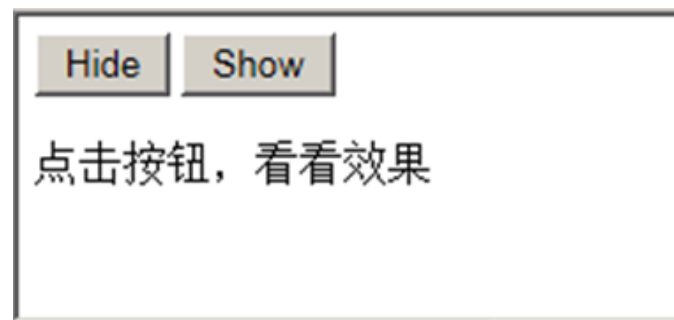
show(duration,[callback]);

hide(duration,[callback]);

其中，duration表示动画执行时间的长短，可以表示速度的字符串，包括slow,normal,fast.也可以是表示时间的整数(毫秒)。callback是可选的回调函数。

【例3-19】 显示和隐藏元素示例，将文字内容隐藏。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
  </head>
  <body>
    <script type="text/javascript">
      $(function() {
        $("input:first").click(function() {
          $("p").hide(300); //隐藏
        });
        $("input:last").click(function() {
          $("p").show(500); //显示
        });
      });
    </script>
    <input type="button" value="Hide">
    <input type="button" value="Show">
    <p>点击按钮，看看效果</p>
  </body>
</html>
```



3. 淡入淡出方法fadeIn()和fadeOut()

jQuery还提供了fadeIn()和fadeOut()这两个实用的方法，其动画效果为淡入淡出，语法与show()和hide()相同：

`fadeIn(duration, [callback]);`

`fadeOut(duration, [callback]);`

【例3-20】 淡入淡出方法应用示例：图片的淡入淡出。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
  </head>
```

```
<body>
  <script type="text/javascript">
    $(function() {
      $("input:eq(0)").click(function() {
        $("img").fadeOut(3000); //淡出
      });
      $("input:eq(1)").click(function() {
        $("img").fadeIn(1000); //淡入
      });
    });
  </script>
  
  <input type="button" value="淡出">
  <input type="button" value="淡入">
</body>
</html>
```



4. 滑动效果slideUp（）和slideDown（）

jQuery还提供了slideUp()和slideDown()来模拟PPT中的类似幻灯片拉帘效果，使用方法与show()和hide()相同。

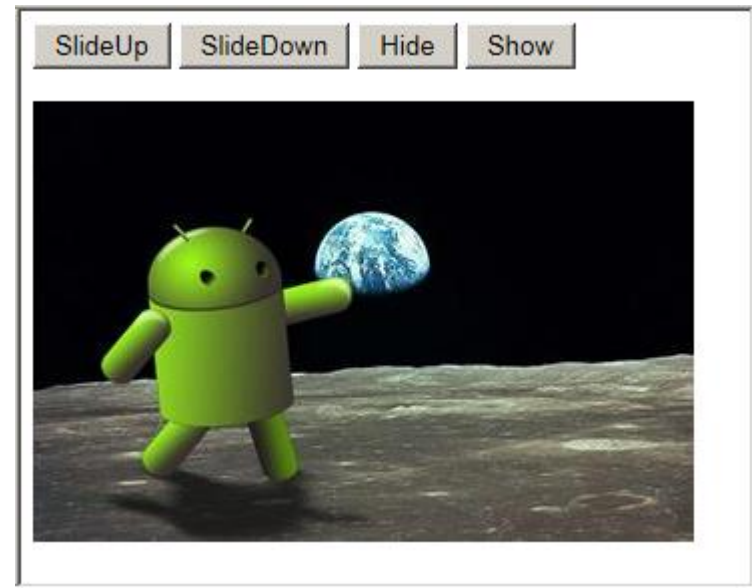
【例3-21】 滑动效果应用示例。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
  </head>
  <body>
```

```

<script type="text/javascript">
    $(function() {
        $("input:eq(0)").click(function() {
            $("div").add("img").slideUp(1000);
        });
        $("input:eq(1)").click(function() {
            $("div").add("img").slideDown(1000);
        });
        $("input:eq(2)").click(function() {
            $("div").add("img").hide(1000);
        });
        $("input:eq(3)").click(function() {
            $("div").add("img").show(1000);
        });
    });
</script>
<input type="button" value="SlideUp">
<input type="button" value="SlideDown">
<input type="button" value="Hide">
<input type="button" value="Show"><br>
<div></div>
</body>
</html>

```



5. 自定义动画

animate()方法能够使开发者自定义动画。

animate(params,[duration],[easing],[callback])

【例3-22】应用animate()方法自定义动画示例。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
  </head>
  <body>
```

```
<style>
    div {
        background-color: #FFFF00;
        height: 40px;
        width: 80px;
        border: 1px solid #000000;
        margin-top: 5px;
        padding: 5px;
        text-align: center;
    }
</style>
```

```
<script type="text/javascript">
```

```
    $(function() {
        $("button").click(function() {
            $("#block").animate({
                opacity: "0.5",
                width: "80%",
                height: "100px",
                borderWidth: "5px",
                fontSize: "30px",
                marginTop: "40px",
                marginLeft: "20px"
            }, 2000);
        });
    });
```

```
    });
</script>
```



//定义 animate() 方法

```
<button id="go">Go>></button>
```

```
<div id="block">动画! </div>
```

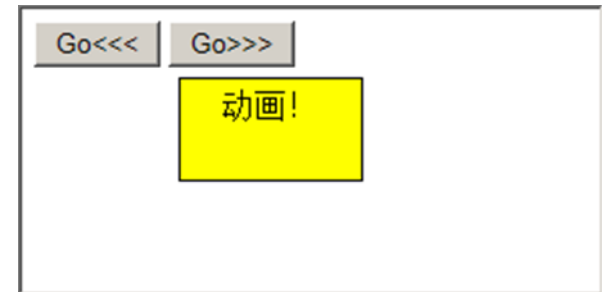
```
</body>
```

```
</html>
```

【例3-23】 使用animate()中的相对移动应用示例。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
  </head>
  <body>
    <style>
      div {
        background-color: #FFFF00;
        height: 40px;
        width: 80px;
        border: 1px solid #000000;
        margin-top: 5px;
        padding: 5px;
        text-align: center;
        position: absolute;
      }
    </style>
```

```
<script type="text/javascript">
    $(function() {
        $("button:first").click(function() {
            $("#block").animate({
                left: "-=80px" //相对左移
            }, 300);
        });
        $("button:last").click(function() {
            $("#block").animate({
                left: "+=80px" //相对右移
            }, 300);
        });
    });
</script>
<button >Go<<<</button>
<button >Go>>>>/button>
<div id="block">动画! </div>
</body>
</html>
```



animate()方法还有另外一种形式，如下所示：

animate(params,options);

【例3-24】 animate()方法另外一种形式应用示例。

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8">
    <!-- 引入 jQuery -->
    <script type="text/javascript" src="jquery-3.1.1.min.js"></script>
  </head>
  <body>
```

```
<style>
  div {
    background-color: #FFFF22;
    width: 100px;
    text-align: center;
    border: 2px solid #000000;
    margin: 3px;
    font-size: 13px;
    font-family: Arial, Helvetica, sans-serif;
  }
  input {
    border: 1px solid #000033;
  }
</style>
```

```
<script type="text/javascript">
    $(function() {
        $("input:eq(0)").click(function() {
            //第一个 animate 与第二个 animate 同时执行，然后再执行第三个
            $("#block1").animate({
                width: "90%"
            }, {
                queue: false,
                duration: 1500
            })
            .animate({
                fontSize: "24px"
            }, 1000)
            .animate({
                borderRightWidth: "20px"
            }, 1000);
        });
        $("input:eq(1)").click(function() {
            //依次执行三个 animate
            $("#block2").animate({
                width: "90%"
            }, 1500)
            .animate({
                fontSize: "24px"
            }, 1000)
            .animate({
                borderRightWidth: "20px"
            }, 1000);
        });
    });

```

```
    $("input:eq(2)").click(function() {  
        $("input:eq(0)").click();  
        $("input:eq(1)").click();  
    });  
    $("input:eq(3)").click(function() {  
        //恢复默认设置  
        $("div").css({  
            width: "",  
            fontSize: "",  
            borderWidth: ""  
        });  
    });  
});  
</script>
```

```
<input type="button" id="go1" value="Block1 动画">  
  <input type="button" id="go2" value="Block2 动画">  
  <input type="button" id="go3" value="同时动画">  
  <input type="button" id="go4" value="重置">  
  <div id="block1">Block1</div>  
  <div id="block2">Block2</div>  
  
</body>  
</html>
```



3.6.2 jQuery实现加入购物车飞入动画效果

本节介绍借助一款基于 jQuery 的动画插件 `jquery.fly.min.js`，点击加入购物车按钮时，实现商品飞入到右侧的购物车中的效果。

可以到fly插件项目的官方网站下载fly插件，其网址为：<https://github.com/amibug/fly>

【例3-25】设计一个将所选择商品飞入购物车的动画效果程序。

1. HTML文件布置页面布局

(1) 载入jQuery库文件和jquery.fly.min.js插件

```
<script src="jquery.js"></script>
```

```
<script src="jquery.fly.min.js"></script>
```

(2) 用四个商品并排布置，每个商品box中包括有商品图片、价格、名称以及加入购物车按钮等信息。



2. CSS提供美化页面效果的样式

3. jQuery实现控制

要实现的效果是，当用户点击“加入购物车”按钮时，当前商品图片会变成一个缩小的圆球，以按钮为起点，向右侧以抛物线的形式飞出，最后落入页面右侧的购物车里，并提示操作成功。在飞出之前，要获取当前商品的图片，然后调用fly插件，之后的抛物线轨迹都是由fly插件完成，只需定义起点和终点等参数即可。

（购物车完整代码详见例程：ex3_25）